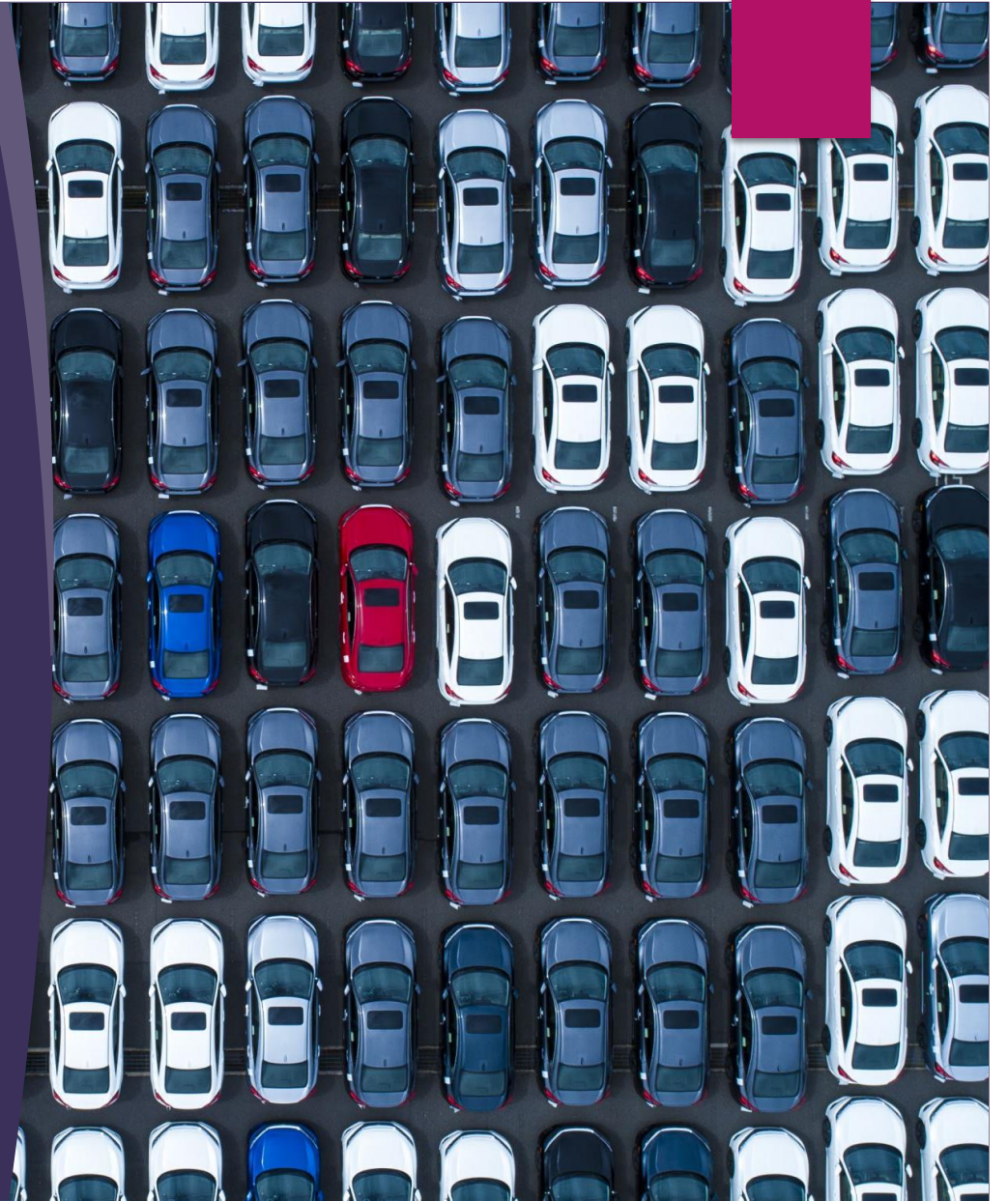


Car Rental Company Management Systems

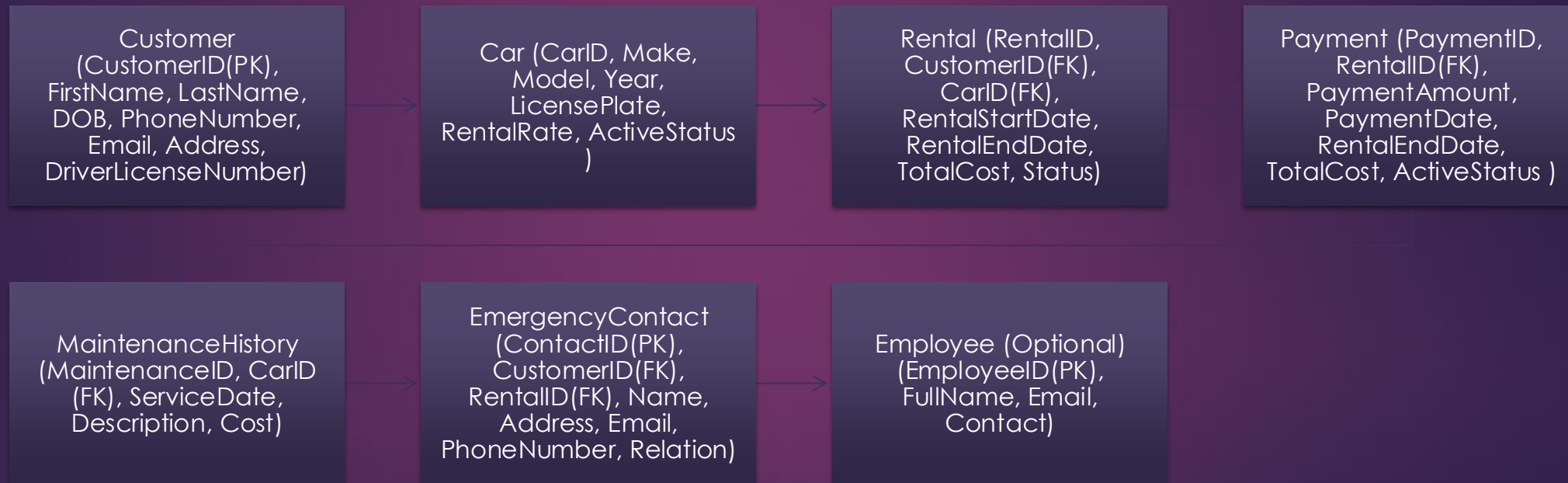
- ▶ ISABEL SILVA
- ▶ ASHIM ALE
- ▶ ALEC DAVIDMAN
- ▶ NABA JAHANZAIB
- ▶ AYELE KOUVIDJIN
- ▶ NICHOLAS OSANI



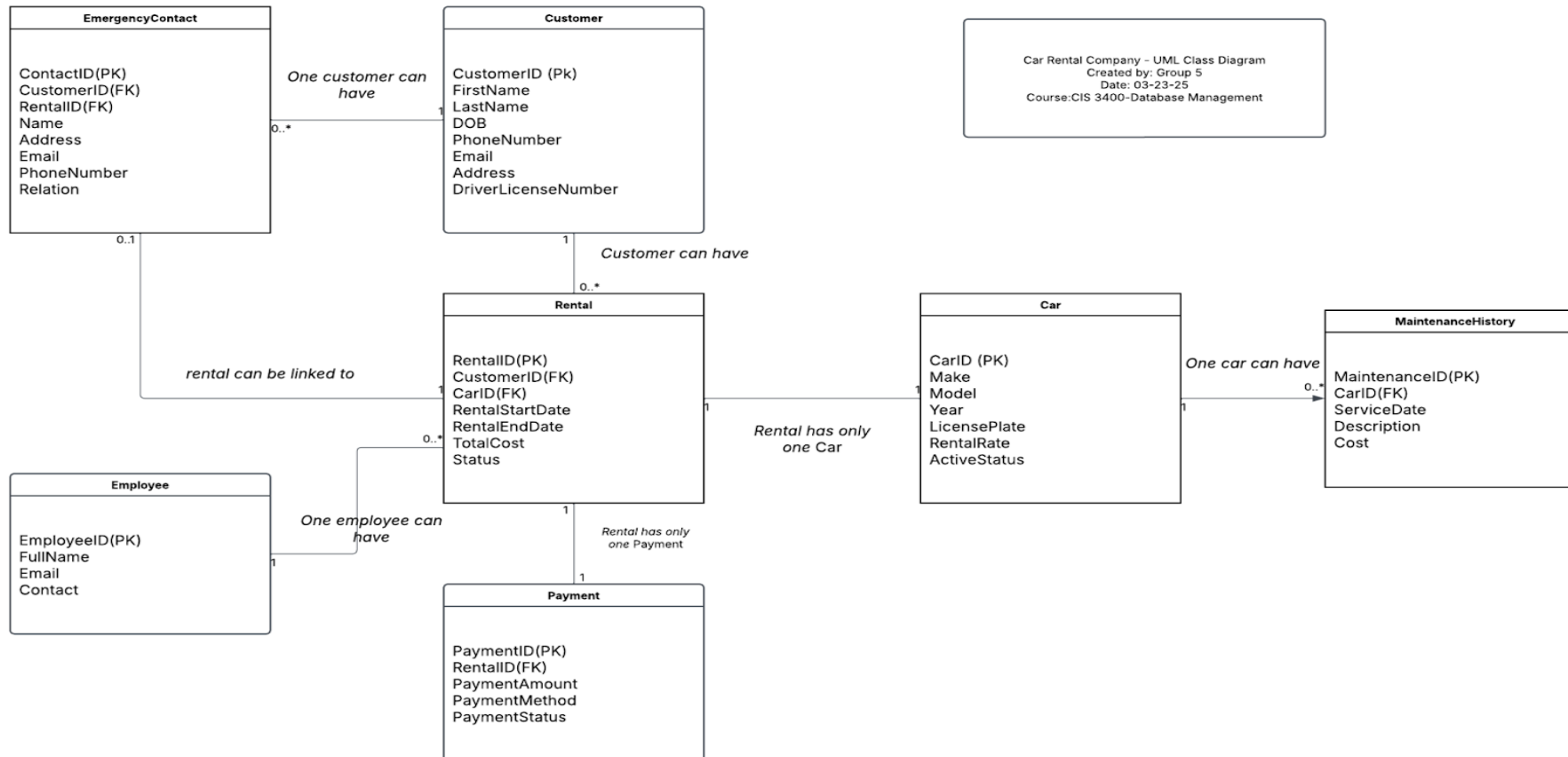
Business Description

- Our group project will focus on a Car Rental Management System. This company has, until now, manually managed its customer information, rental reservations, and car availability. The company had been using spreadsheets and paper-based logs. This system has been dated and has to be changed as oftentimes there would be significant inconsistencies ranging from double bookings, lost customer records/information, and keeping track of car availability. In order to address these issues, we have proposed to develop a database system that will manage customers, cars, rental reservations, and payments.

Initial List of Entities



E-R Model



Logical Modeling



Customer:

Schema:
Customer(CustomerID (PK),
FirstName, LastName, DOB,
PhoneNumber, Email,
Address,
DriverLicenseNumber)

Notes: CustomerID is the
unique primary key; it links
to rental reservations.



Car:

Schema: Car(CarID (PK),
Make, Model, Year,
LicensePlate, RentalRate,
ActiveStatus)

Notes: CarID is the primary
key. LicensePlate could
serve as an alternate key.



Rental:

Schema:
Rental(RentalID (PK),
CustomerID (FK), CarID
(FK), RentalStartDate,
RentalEndDate, TotalCost,
Status)

Notes: CustomerID and
CarID are foreign keys that
reference Customer and
Car, respectively.

Logical Modeling



Payment:

Schema:

Payment(PaymentID (PK),
RentalID (FK),
PaymentAmount,
PaymentDate,
RentalEndDate, TotalCost,
ActiveStatus)

Notes: PaymentID is the
unique primary key;
RentalID is a foreign key
referencing Rental.



MaintenanceHistory:

Schema:

MaintenanceHistory(Maint
enanceID (PK), CarID (FK),
ServiceDate, Description,
Cost)

Notes: CarID references
Car.



EmergencyContact:

Schema:

EmergencyContact(Conta
ctID (PK), CustomerID (FK),
RentalID (FK), Name,
Address, Email,
PhoneNumber, Relation)

Notes: ContactID is the
primary key; CustomerID
and RentalID are foreign
keys referencing Customer
and Rental, respectively.

Normalization

- Customer(CustomerID (PK), FirstName, LastName, DOB, PhoneNumber, Email, Address, DriverLicenseNumber)
- Car(CarID (PK), Make, Model, Year, LicensePlate, RentalRate, ActiveStatus)
- Rental(RentalID (PK), CustomerID (FK), CarID (FK), RentalStartDate, RentalEndDate, TotalCost, Status)
- Payment(PaymentID (PK), RentalID (FK), PaymentAmount, PaymentDate, ActiveStatus)
 - (Removed RentalEndDate, TotalCost to eliminate transitive dependency)
- MaintenanceHistory(MaintenanceID (PK), CarID (FK), ServiceDate, Description, Cost)
- EmergencyContact(ContactID (PK), RentalID (FK), Name, Address, Email, PhoneNumber, Relation)
 - (Removed CustomerID to eliminate transitive dependency)
- Employee(EmployeeID (PK), FullName, Email, Contact)

Creating DB Schema Using SQL Queries

- The following SQL code creates the 7 tables and adds the PRIMARY KEY constraint to each one:
Customer Table: Stores customer personal and license information.

```
CREATE TABLE Customer (  
    CustomerID NUMBER NOT NULL,  
    FirstName VARCHAR(50) NOT NULL,  
    LastName VARCHAR(50) NOT NULL,  
    DOB DATE NOT NULL,  
    PhoneNumber VARCHAR(15) NOT NULL,  
    Email VARCHAR(50) NOT NULL,  
    Address VARCHAR(100) NOT NULL,  
    DriverLicenseNumber VARCHAR(15) NOT NULL
```

- Car Table: Stores vehicle details and rental rate.
Note: LicensePlate could serve as an alternate key.

```
CREATE TABLE Car (  
    CarID NUMBER NOT NULL,  
    Make VARCHAR(30) NOT NULL,  
    Model VARCHAR(30) NOT NULL,  
    [Year] NUMBER NOT NULL,  
    LicensePlate VARCHAR(10) NOT NULL,  
    RentalRate NUMBER NOT NULL,  
    ActiveStatus VARCHAR(10) NOT NULL
```


Creating DB Schema Using SQL Queries

Rental Table: Records each car rental transaction linking a customer to a car, with rental period, cost, and status.

```
CREATE TABLE Rental (  
    RentalID    NUMBER    NOT NULL,  
    CustomerID  NUMBER    NOT NULL,  
    CarID       NUMBER    NOT NULL,  
    RentalStartDate DATE    NOT NULL,  
    RentalEndDate DATE    NOT NULL,  
    TotalCost   NUMBER    NOT NULL,  
    Status      VARCHAR(10) NOT NULL  
);
```

Payment Table: Records payments associated with rentals (normalized to remove redundant RentalEndDate and TotalCost)

```
CREATE TABLE Payment (  
    PaymentID   NUMBER    NOT NULL,  
    RentalID    NUMBER    NOT NULL,  
    PaymentAmount NUMBER    NOT NULL,  
    PaymentDate DATE    NOT NULL,  
    ActiveStatus VARCHAR(10) NOT NULL  
);
```

MaintenanceHistory Table: Tracks maintenance events for cars (date, description, cost).

```
CREATE TABLE MaintenanceHistory (  
    MaintenanceID NUMBER    NOT NULL,  
    CarID          NUMBER    NOT NULL,  
    ServiceDate    DATE    NOT NULL,  
    Description     VARCHAR(100) NOT NULL,  
    Cost           NUMBER    NOT NULL  
);
```

Creating DB Schema Using SQL Queries

EmergencyContact Table: Stores emergency contact info for rentals (normalized to remove direct Customer reference). Each rental may have one or more emergency contacts.

```
CREATE TABLE EmergencyContact (  
    ContactID NUMBER NOT NULL,  
    RentalID NUMBER NOT NULL,  
    Name VARCHAR(50) NOT NULL,  
    Address VARCHAR(100) NOT NULL,  
    Email VARCHAR(50) NOT NULL,  
    PhoneNumber VARCHAR(15) NOT NULL,  
    Relation VARCHAR(20) NOT NULL  
);
```

Employee Table: Stores employee contact information (not directly linked to rentals in this design).

```
CREATE TABLE Employee (  
    EmployeeID NUMBER NOT  
    NULL,  
    FullName VARCHAR(50) NOT  
    NULL,  
    Email VARCHAR(50) NOT  
    NULL,  
    Contact VARCHAR(15) NOT  
    NULL  
);
```

Adding Primary Keys: Each table's primary key is added as a constraint to uniquely identify records.

```
ALTER TABLE Customer ADD CONSTRAINT  
pk_Customer PRIMARY KEY (CustomerID);
```

```
ALTER TABLE Car ADD CONSTRAINT pk_Car  
PRIMARY KEY (CarID);
```

```
ALTER TABLE Rental ADD CONSTRAINT pk_Rental  
PRIMARY KEY (RentalID);
```

```
ALTER TABLE Payment ADD CONSTRAINT  
pk_Payment PRIMARY KEY (PaymentID);
```

```
ALTER TABLE MaintenanceHistory ADD  
CONSTRAINT pk_Maintenance PRIMARY KEY  
(MaintenanceID);
```

```
ALTER TABLE EmergencyContact ADD CONSTRAINT  
pk_Contact PRIMARY KEY (ContactID);
```

```
ALTER TABLE Employee ADD CONSTRAINT  
pk_Employee PRIMARY KEY (EmployeeID);
```

Creating DB Schema Using SQL Queries

► Adding Foreign Keys: We now add foreign key constraints to link tables together. These ensure that references (e.g. Rental.CustomerID) correspond to existing records in the related table (e.g. Customer.CustomerID), preventing orphan records:

```
ALTER TABLE Rental
```

```
    ADD CONSTRAINT fk_Rental_Customer
```

```
    FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID);
```

```
ALTER TABLE Rental
```

```
    ADD CONSTRAINT fk_Rental_Car
```

```
    FOREIGN KEY (CarID) REFERENCES Car(CarID);
```

```
ALTER TABLE Payment
```

```
    ADD CONSTRAINT fk_Payment_Rental
```

```
    FOREIGN KEY (RentalID) REFERENCES Rental(RentalID);
```

```
ALTER TABLE MaintenanceHistory
```

```
    ADD CONSTRAINT fk_Maint_Car
```

```
    FOREIGN KEY (CarID) REFERENCES Car(CarID);
```

```
ALTER TABLE EmergencyContact
```

```
    ADD CONSTRAINT fk_Contact_Rental
```

```
    FOREIGN KEY (RentalID) REFERENCES Rental(RentalID);
```

```

INSERT INTO Customer VALUES
(1, "John", "Doe", #1985-05-15#, "212-555-1111", "johndoe@example.com", "123 Main St, New York, NY", "JD123456");

INSERT INTO Customer VALUES
(2, "Jane", "Smith", #1990-10-30#, "212-555-2222", "janesmith@example.com", "456 Oak Ave, Los Angeles, CA", "JS123457");

INSERT INTO Customer VALUES
(3, "Bob", "Martin", #1975-02-20#, "212-555-3333", "bobmartin@example.com", "789 Pine Rd, Houston, TX", "BM123458");

INSERT INTO Customer VALUES (4, "Alice", "Brown", #1995-12-05#, "718-555-4444", "alicebrown@example.com", "321 Maple Dr, Chicago, IL", "AB123459");

INSERT INTO Customer VALUES
(5, "Carol", "White", #1988-07-11#, "718-555-5555", "carolwhite@example.com", "654 Elm St, Miami, FL", "CW123460");

INSERT INTO Customer VALUES
(6, "Derek", "Black", #2000-01-01#, "917-555-6666", "derekblack@example.com", "1010 Cedar Ln, Boston, MA", "DB123461");

```

Car		Customer	EmergencyContact	Employee	MaintenanceHistory	Payment	Rental		
	CustomerID	FirstName	LastName	DOB	PhoneNum	Email	Address	DriverLicens	Click to Add
+	1	John	Doe	5/15/1985	212-555-1111	johndoe@exar	123 Main St, Ne	JD123456	
+	2	Jane	Smith	10/30/1990	212-555-2222	janesmith@ex	456 Oak Ave, Lo	JS123457	
+	3	Bob	Martin	2/20/1975	212-555-3333	bobmartin@ex	789 Pine Rd, Ho	BM123458	
+	4	Alice	Brown	12/5/1995	718-555-4444	alicebrown@e	321 Maple Dr, C	AB123459	
+	5	Carol	White	7/11/1988	718-555-5555	carolwhite@e	654 Elm St, Mia	CW123460	
+	6	Derek	Black	1/1/2000	917-555-6666	derekblack@e	1010 Cedar Ln,	DB123461	

Data Insertion (INSERT Statements):

Customer Table: Six customers with unique personal details. Includes one customer (ID 6) who hasn't rented yet (to demonstrate storing prospects as well as active renters).

```

INSERT INTO Car VALUES
(1, "Toyota", "Camry", 2019, "ABC123", 45, "Active");

INSERT INTO Car VALUES
(2, "Honda", "Civic", 2020, "XYZ789", 40, "Active");

INSERT INTO Car VALUES
(3, "Ford", "Fiesta", 2018, "HJK456", 30, "Inactive");

INSERT INTO Car VALUES
(4, "Tesla", "Model 3", 2021, "EV1234", 80, "Active");

INSERT INTO Car VALUES
(5, "BMW", "X5", 2022, "BMWX5", 100, "Active");

INSERT INTO Car VALUES
(6, "Mercedes", "S-Class", 2021, "MERC999", 200, "Active");

```

Car		Customer	EmergencyContact	Employee	MaintenanceHistory	Payment	Rental	
	CarID	Make	Model	Year	LicensePlate	RentalRate	ActiveStatus	Click to Add
+	1	Toyota	Camry	2019	ABC123	45	Active	
+	2	Honda	Civic	2020	XYZ789	40	Active	
+	3	Ford	Fiesta	2018	HJK456	30	Inactive	
+	4	Tesla	Model 3	2021	EV1234	80	Active	
+	5	BMW	X5	2022	BMWX5	100	Active	
+	6	Mercedes	S-Class	2021	MERC999	200	Active	
*								

Car Table: Six cars in the fleet with various makes/models. ActiveStatus indicates if the car is available for rentals. (CarID 6 is a high-end vehicle that has not been rented yet, demonstrating an underutilized asset.)

```

INSERT INTO Rental VALUES
(1, 1, 1, #2025-04-10#, #2025-04-14#, 225, "Completed");

INSERT INTO Rental VALUES
(2, 2, 2, #2025-01-05#, #2025-01-07#, 120, "Completed");

INSERT INTO Rental VALUES
(3, 4, 2, #2025-04-20#, #2025-04-26#, 280, "Completed");

INSERT INTO Rental VALUES
(4, 3, 3, #2025-04-25#, #2025-05-02#, 240, "Overdue");

INSERT INTO Rental VALUES
(5, 1, 4, #2025-05-03#, #2025-05-08#, 480, "Active");

INSERT INTO Rental VALUES
(6, 5, 1, #2025-03-01#, #2025-03-04#, 180, "Completed");

INSERT INTO Rental VALUES
(7, 4, 5, #2025-06-01#, #2025-06-05#, 500, "Reserved");

```

Car		Customer		EmergencyContact		Employee		MaintenanceHistory		Payment		Rental	
	RentalID	CustomerID	CarID	RentalStartC	RentalEndD	TotalCost	Status	Click to Add					
+	1	1	1	4/10/2025	4/14/2025	225	Completed						
+	2	2	2	1/5/2025	1/7/2025	120	Completed						
+	3	4	2	4/20/2025	4/26/2025	280	Completed						
+	4	3	3	4/25/2025	5/2/2025	240	Overdue						
+	5	1	4	5/3/2025	5/8/2025	480	Active						
+	6	5	1	3/1/2025	3/4/2025	180	Completed						
+	7	4	5	6/1/2025	6/5/2025	500	Reserved						
*													

Rental Table: Seven rental records linking customers to cars over specific date ranges. This includes various scenarios: Completed rentals (returned on time), An overdue rental (RentalID 4) where the car was not returned by the due date, A currently active rental in progress (RentalID 5), A future reservation (RentalID 7, not yet started). Each rental has a Status (Completed, Overdue, Active, or Reserved) and a TotalCost. (Note: Rental 6 uses Car1 again, showing a car rented by multiple customers at different times.)

```
INSERT INTO Payment VALUES  
(1, 1, 225, #2025-04-14#, "Active");
```

```
INSERT INTO Payment VALUES  
(2, 2, 120, #2025-01-07#, "Active");
```

```
INSERT INTO Payment VALUES  
(3, 3, 100, #2025-04-20#, "Active");
```

```
INSERT INTO Payment VALUES  
(4, 3, 180, #2025-04-26#, "Active");
```

```
INSERT INTO Payment VALUES  
(5, 4, 100, #2025-04-25#, "Active");
```

```
INSERT INTO Payment VALUES  
(6, 5, 200, #2025-05-03#, "Active");
```

```
INSERT INTO Payment VALUES  
(7, 6, 180, #2025-03-04#, "Active");
```

```
INSERT INTO Payment VALUES  
(8, 7, 150, #2025-05-01#, "Active");
```

Car	Customer	EmergencyContact	Employee	MaintenanceHistory	Payment	Rental
PaymentID	RentalID	PaymentAm	PaymentDat	ActiveStatus	Click to Add	
1	1	225	4/14/2025	Active		
2	2	120	1/7/2025	Active		
3	3	100	4/20/2025	Active		
4	3	180	4/26/2025	Active		
5	4	100	4/25/2025	Active		
6	5	200	5/3/2025	Active		
7	6	180	3/4/2025	Active		
8	7	150	5/1/2025	Active		
*						

Payment Table: Records payments for each rental. Most rentals are paid in full upon completion, but we also demonstrate partial payments:
For the overdue rental (ID 4) and the active rental (ID 5), only an initial deposit has been paid so far.
For one completed rental (ID 3), the customer paid in two installments (deposit and final payment), showing multiple payments for a single rental.
All amounts are in whole dollars for simplicity, and PaymentDate indicates when the payment was made. ActiveStatus is set to "Active" for all entries here to indicate each payment record is valid (none have been canceled).


```
INSERT INTO MaintenanceHistory VALUES  
(1, 1, #2024-08-15#, "Oil Change", 100);
```

```
INSERT INTO MaintenanceHistory VALUES  
(2, 1, #2025-02-10#, "Brake Pad Replacement", 300);
```

```
INSERT INTO MaintenanceHistory VALUES  
(3, 2, #2025-03-01#, "Tire Replacement", 150);
```

```
INSERT INTO MaintenanceHistory VALUES  
(4, 3, #2024-12-20#, "Engine Repair", 500);
```

```
INSERT INTO MaintenanceHistory VALUES  
(5, 4, #2025-04-01#, "Tire Rotation", 60);
```

Car	Customer	EmergencyContact	Employee	MaintenanceHistory	Payment	Rental
Maintenano	CarID	ServiceDate	Description	Cost	Click to Add	
	1	1	8/15/2024	Oil Change	100	
	2	1	2/10/2025	Brake Pad Repl	300	
	3	2	3/1/2025	Tire Replaceme	150	
	4	3	12/20/2024	Engine Repair	500	
	5	4	4/1/2025	Tire Rotation	60	
*						

MaintenanceHistory Table: Five maintenance records for various cars. This shows routine maintenance and repairs: Car1 had multiple services (oil change, brake pads), Car2, Car3, Car4 had at least one maintenance each, Car5 and Car6 currently have no records (e.g., Car6 might be new with no maintenance yet).

```
INSERT INTO Employee VALUES  
(1, "Michael Johnson", "michael.johnson@carrental.com", "646-555-1001");
```

```
INSERT INTO Employee VALUES  
(2, "Sarah Carter", "sarah.carter@carrental.com", "646-555-1002");
```

```
INSERT INTO Employee VALUES  
(3, "Emily Davis", "emily.davis@carrental.com", "646-555-1003");
```

```
INSERT INTO Employee VALUES  
(4, "David Lee", "david.lee@carrental.com", "646-555-1004");
```

Car	Customer	EmergencyContact	Employee	MaintenanceHistory	Payment	Rental
EmployeeID	FullName	Email	Contact	Click to Add		
1	Michael Johnson	michael.johnson@carrental.com	646-555-1001			
2	Sarah Carter	sarah.carter@carrental.com	646-555-1002			
3	Emily Davis	emily.davis@carrental.com	646-555-1003			
4	David Lee	david.lee@carrental.com	646-555-1004			
*						

Employee Table: Four employees of the rental company, with their contact information.

```

INSERT INTO EmergencyContact VALUES
(1, 1, "Mary Doe", "123 Main St, New York, NY", "mary.doe@example.com", "212-555-1111", "Spouse");

INSERT INTO EmergencyContact VALUES
(2, 2, "Tom Smith", "987 Pine Rd, Sacramento, CA", "tom.smith@example.com", "212-555-7777", "Father");

INSERT INTO EmergencyContact VALUES
(3, 3, "George Brown", "321 Maple Dr, Chicago, IL", "george.brown@example.com", "312-555-8888", "Father");

INSERT INTO EmergencyContact VALUES
(4, 4, "Linda Martin", "789 Pine Rd, Houston, TX", "linda.martin@example.com", "713-555-9999", "Mother");

INSERT INTO EmergencyContact VALUES
(5, 4, "Alan Martin", "789 Pine Rd, Houston, TX", "alan.martin@example.com", "713-555-0000", "Father");

INSERT INTO EmergencyContact VALUES
(6, 5, "Mary Doe", "123 Main St, New York, NY", "mary.doe@example.com", "212-555-1111", "Spouse");

INSERT INTO EmergencyContact VALUES
(7, 6, "Sam White", "654 Elm St, Miami, FL", "sam.white@example.com", "305-555-1234", "Husband");

INSERT INTO EmergencyContact VALUES
(8, 7, "Susan Miller", "1000 Lake Ave, Chicago, IL", "susan.miller@example.com", "312-555-7777", "Friend");

```

1	1	Mary Doe	123 Main St, New York, NY	mary.doe@example.com	212-555-1111	Spouse	
2	2	Tom Smith	987 Pine Rd, Sacramento, CA	tom.smith@example.com	212-555-7777	Father	
3	3	George Brown	321 Maple Dr, Chicago, IL	george.brown@example.com	312-555-8888	Father	
4	4	Linda Martin	789 Pine Rd, Houston, TX	linda.martin@example.com	713-555-9999	Mother	
5	4	Alan Martin	789 Pine Rd, Houston, TX	alan.martin@example.com	713-555-0000	Father	
6	5	Mary Doe	123 Main St, New York, NY	mary.doe@example.com	212-555-1111	Spouse	
7	6	Sam White	654 Elm St, Miami, FL	sam.white@example.com	305-555-1234	Husband	
8	7	Susan Miller	1000 Lake Ave, Chicago, IL	susan.miller@example.com	312-555-7777	Friend	

EmergencyContact Table: Eight emergency contact entries. Each rental has at least one emergency contact on file, and one rental (ID 4) has two contacts. This demonstrates the one-to-many relationship between Rental and EmergencyContact: The same person can appear as a contact for multiple rentals (e.g., Mary Doe is listed for John Doe's rentals 1 and 5). Different rentals can have contacts with various relations (spouse, parent, friend, etc.).

Average Rental Duration (days)	List Overdue Rentals
SELECT Rental.RentalID, Customer.FirstName, Customer.LastName, Rental.RentalEndDate, Rental.Status FROM Rental INNER JOIN Customer ON Rental.CustomerID = Customer.CustomerID WHERE Rental.RentalEndDate < Date() AND Rental.Status <> "Completed";	

Average Rental Duration (days)

List Overdue Rentals

RentalID	FirstName	LastName	RentalEndDate	Status
4	Bob	Martin	5/2/2025	Overdue
5	John	Doe	5/8/2025	Active
*				

Query 1: List Overdue Rentals and Customers

Purpose: Identify rentals that are overdue (past their end date and not completed) along with the customer who has the car. This information helps staff quickly see which rentals need attention (e.g. follow-up calls to the customer or their emergency contacts to retrieve the car and close the rental).

Explanation: This query finds any rental where the end date is before today (DATE() in Access returns the current date) and the status is not marked "Completed" (meaning the car hasn't been returned on time). It joins the Rental table with Customer to retrieve the customer's name for each overdue rental. Management can use this to generate an overdue rentals report and take immediate action on late returns.

Average Rental Duration (days)	List Overdue Rentals	Never Rented (underutilized)
<pre> SELECT Car.Make, Car.Model, Car.[Year], Car.LicensePlate FROM Car WHERE Car.CarID NOT IN (SELECT Rental.CarID FROM Rental); </pre>		

Average Rental Duration (days) X		List Overdue Rentals X		Never Rented (underutilized) X	
Make	Model	Year	LicensePlate		
Mercedes	S-Class	2021	MERC999		
*					

Query 2: Find Underutilized Cars (Never Rented)

Purpose: Determine which cars in the fleet have never been rented. These underutilized vehicles represent wasted inventory – the company can decide to market them more aggressively or remove them from the fleet to save costs.

Explanation: This query uses a subquery to find cars that have no entries in the Rental table. The NOT IN clause filters for cars whose CarID is not found among any rental records. The result will list cars (by make, model, year, and plate) that have never been rented. These are clear candidates for review – for example, in our sample data Car6 (a costly luxury model) might appear here, indicating it has yet to generate any rental revenue.

Average Rental Duration (days) X

SELECT AVG(DateDiff("d", RentalStartDate, RentalEndDate) + 1) AS AvgRentalDurationDays
FROM Rental;

Average Rental Duration (days) X	
AvgRentalDurationDays	
5.42857142857143	

Query 3: Calculate Average Rental Duration

Purpose: Compute the average rental duration (in days) for all completed rentals. This gives a broader operational insight into customer behavior – knowing the typical rental length helps in fleet planning, rate policy (e.g., discounts for weekly rentals), and predicting maintenance schedules.

Explanation: This query uses the DateDiff function to calculate the number of days between the start and end dates of each rental. We add 1 to count both the pickup and return days as part of the rental period. By taking the average of this difference across all rentals, we get the average rental length in days. For instance, if most customers rent for about 5 days on average, the company might tailor its pricing packages (weekly rates, weekend specials, etc.) accordingly.

This metric helps in understanding usage patterns and optimizing operations over time.



Thank You For Your Attention!
Questions / Comments / Concerns?